

IPSignal Fraud Analysis - Updated April 2026 Code Package

Client-facing update with filtered rule recommendations, integration-point logic, Score 3 inclusion, triggered transactions, and deduplicated cumulative impact

Purpose

This document updates the previous April 2026 IPSignal analysis code based on the latest review findings. The key objective is to prevent weak or misleading rules from appearing as recommendations and to make the final output more client-ready.

- Use April 2026 only as the analysis window.
- Remove client-facing references to IPv6 and keep the analysis focused on the IPSignal-scored universe.
- Include Score 3 in score performance and rule testing.
- Separate layered rules from velocity-only rules.
- Add triggered transactions, triggered transaction share, triggered frauds, legitimate transactions touched, and friction metrics.
- Filter out rules with zero fraud, extremely low volume, excessive friction, threshold=1, or weak lift.
- Evaluate RP1, RP2, and RP3 separately because RP3 is not always the highest-risk tier.
- Generate cumulative rule impact only from qualified rules, and include incremental contribution per additional rule.

Major Changes Versus Prior Code

Area	Previous behavior	Updated behavior
Rule filtering	Displayed all candidate rules	Keeps only rules meeting minimum txns, frauds, lift, friction, and share criteria
Velocity rules	Could include threshold=1 rules	Drops threshold=1 and rules that trigger too much traffic
Residential proxy	Focused heavily on RP3	Tests RP1, RP2, and RP3 separately
Score 3	Mentioned but not fully operationalized	Included in score metrics and layered rules
Integration points	Mixed quality of recommendations	Only recommends rules that qualify within each integration point
Cumulative impact	Could include weak rules	Uses only qualified recommendations and reports incremental contribution
Charts	Could generate empty plots	Skips empty/no-value plots and prints a clear note

Recommendation Logic Included in the Code

The updated logic applies explicit qualification criteria before a rule can be recommended. These criteria prevent rules with no observed fraud, very small samples, or excessive traffic impact from being shown as client-ready recommendations.

```
MIN_TRIGGERED_TXNS = 100
MIN_TRIGGERED_FRAUDS = 10
MIN_FRAUD_LIFT = 1.5
MAX_FRICTION_RATE = 0.10
MAX_TRIGGERED_SHARE = 0.30
```

The final recommendations are split into two groups: layered rules and velocity-only rules. Layered rules combine score, reason families, and sometimes velocity. Velocity-only rules are retained because the results showed they are stronger for some integration points, especially receiver flows.

Client-Facing Interpretation Guidance

- Score 3 should be positioned as a small but high-risk segment. It has value as part of layered rules, not as a standalone broad decision rule.
- Score 5 captures fraud by volume, but it is too broad to use alone for deny decisions.
- Anonymous / Privacy reason family is the strongest precision signal and should be prioritized in review or step-up strategies.
- Velocity rules should be recommended by integration point, not as generic universal rules.
- Receiver flows should be evaluated with velocity-only rules where layered score + RP rules do not capture fraud.
- Application currently needs refinement if candidate rules fail the minimum criteria.
- Login supports layered rules and velocity rules, but cumulative impact should be deduplicated.

Complete Updated Python + DuckDB Code

Copy the following script into a notebook or Python file. Update PARQUET_PATH before running. The script creates CSV outputs and PNG visualizations in the output directory.

```
# =====
# IPSignal April 2026 Client-Facing Analysis - Updated Version
# Updated to incorporate latest review feedback:
# - April 2026 analysis window only
# - Client-facing outputs remove IPv6 references
# - Include Score 3 in score and rule analysis
# - Separate layered rules vs velocity-only rules
# - Add triggered transactions, triggered share, legitimate txns touched
# - Add minimum rule qualification filters
# - Evaluate RP1/RP2/RP3 separately; do not assume RP3 is always highest risk
# - Exclude zero-fraud, excessive-friction, threshold=1, and empty rules
# - Generate deduplicated cumulative and incremental rule impact
# =====

import duckdb
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# -----
# 1. Configuration
# -----
DB_PATH = "ipsignal_fraud_analysis.duckdb"
PARQUET_PATH = "/path/to/parquet/files/*.parquet" # <-- update this path
OUTPUT_DIR = Path("client_facing_april2026_outputs_v2")
OUTPUT_DIR.mkdir(exist_ok=True)

ANALYSIS_START = "2026-04-01"
ANALYSIS_END = "2026-05-01"

# Minimum rule qualification criteria
MIN_TRIGGERED_TXNS = 100
MIN_TRIGGERED_FRAUDS = 10
MIN_FRAUD_LIFT = 1.5
MAX_FRICTION_RATE = 0.10
MAX_TRIGGERED_SHARE = 0.30
MAX_RULES_PER_INTEGRATION = 5

con = duckdb.connect(DB_PATH)
con.execute("PRAGMA threads=8;")
con.execute("PRAGMA memory_limit='8GB';")
con.execute("PRAGMA temp_directory='/tmp/duckdb_temp';")

plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["axes.titlesize"] = 14
plt.rcParams["axes.labelsize"] = 11
plt.rcParams["xtick.labelsize"] = 10
plt.rcParams["ytick.labelsize"] = 10

def save_plot(filename):
    path = OUTPUT_DIR / filename
    plt.tight_layout()
    plt.savefig(path, dpi=300, bbox_inches="tight")
    plt.show()
    print(f"Saved: {path}")

def esc_sql(value: str) -> str:
    return value.replace("'", "''")

# -----
# 2. Load parquet and normalize column access
# -----
con.execute(f"""
CREATE OR REPLACE VIEW raw_data AS
SELECT *
FROM read_parquet('{PARQUET_PATH}', union_by_name = true, filename = true);
""")

raw_cols = con.execute("DESCRIBE raw_data").df()["column_name"].tolist()
raw_cols_upper = {c.upper(): c for c in raw_cols}

def pick_col(candidates, required=True):
    """Return a quoted DuckDB identifier for the first existing column."""
    for c in candidates:
        if c.upper() in raw_cols_upper:
            return f'"{raw_cols_upper[c.upper()]}"'
    if required:
        raise ValueError(f"Missing required column. Tried: {candidates}")
    return None

def col_or_null(candidates, alias, cast_type=None):
```

```

col = pick_col(candidates, required=False)
if col is None:
    return f"NULL AS {alias}"
if cast_type:
    return f"TRY_CAST({col} AS {cast_type}) AS {alias}"
return f"{col} AS {alias}"

# Essential columns
C_UUID = pick_col(["UUID", "uuid"])
C_TS = pick_col(["MSG_TIMESTAMP", "msg_timestamp"])
C_MSG_DAY = pick_col(["MSG_DAY", "msg_day"], required=False)
C_RESULT = pick_col(["RESULT", "result"])
C_FRAUD = pick_col(["FRAUD_DIRECT", "fraud_direct"])
C_SCORE = pick_col(["IP_SIGNAL_SCORE_1", "ipsignal_score_1", "IP_SIGNAL_SCORE"])
C_LAST_SCORE = pick_col(["IP_SIGNAL_LAST_SCORE_1", "ipsignal_last_score_1", "IP_SIGNAL_LAST_SCORE"],
    required=False)
C_INTEGRATION = pick_col(["INTEGRATION_POINT_NAME", "integration_point_name", "RULE_SET_NAME"],
    required=True)

# Optional entity columns
C_DEVICE = pick_col(["DEVICE_ID", "device_id"], required=False)
C_ACCOUNT = pick_col(["ACCOUNT_CODE", "account_code"], required=False)
C_IP = pick_col(["REAL_IP_ADDRESS_1", "REAL_IPADDRESS", "REAL_IP_ADDRESS", "real_ipaddress"],
    required=False)

# Reason code columns
reason_cols = []
for i in range(1, 7):
    col = pick_col([f"CODE_{i}", f"code_{i}"], required=False)
    reason_cols.append(col if col else "NULL")

C_RESULT_CODES = pick_col(["RESULT_CODES_1", "result_codes_1", "RESULT_CODES"], required=False)

# -----
# 3. Create April 2026 client-facing scored universe
# -----
# This universe intentionally does not segment by IP version for the client-facing version.
# It contains transactions with a valid IPSignal score during April 2026.

msg_day_expr = f"TRY_CAST({C_MSG_DAY} AS DATE)" if C_MSG_DAY else f"TRY_CAST({C_TS} AS DATE)"
last_score_expr = f"TRY_CAST({C_LAST_SCORE} AS INTEGER)" if C_LAST_SCORE else "NULL"
device_expr = C_DEVICE if C_DEVICE else "NULL"
account_expr = C_ACCOUNT if C_ACCOUNT else "NULL"
ip_expr = C_IP if C_IP else "NULL"
result_codes_expr = C_RESULT_CODES if C_RESULT_CODES else "NULL"

con.execute(f"""
CREATE OR REPLACE TABLE april_base AS
SELECT
    {C_UUID} AS uuid,
    TRY_CAST({C_TS} AS TIMESTAMP) AS ts,
    {msg_day_expr} AS msg_day,

    {device_expr} AS device_id,
    {account_expr} AS account_code,
    {ip_expr} AS real_ipaddress,

    LOWER(TRIM(CAST({C_RESULT} AS VARCHAR))) AS result,
    COALESCE(TRY_CAST({C_FRAUD} AS INTEGER), 0) AS fraud_direct,

    TRY_CAST({C_SCORE} AS INTEGER) AS ipsignal_score,
    {last_score_expr} AS ipsignal_last_score,

    {result_codes_expr} AS result_codes,
    {reason_cols[0]} AS code_1,
    {reason_cols[1]} AS code_2,
    {reason_cols[2]} AS code_3,
    {reason_cols[3]} AS code_4,
    {reason_cols[4]} AS code_5,
    {reason_cols[5]} AS code_6,

    CAST({C_INTEGRATION} AS VARCHAR) AS integration_point

FROM raw_data
WHERE TRY_CAST({C_TS} AS TIMESTAMP) >= TIMESTAMP '{ANALYSIS_START}'
    AND TRY_CAST({C_TS} AS TIMESTAMP) < TIMESTAMP '{ANALYSIS_END}'
    AND TRY_CAST({C_SCORE} AS INTEGER) BETWEEN 1 AND 5;
""")

# -----
# 4. Baseline
# -----
baseline = con.execute("""
SELECT
    COUNT(*) AS txns,
    SUM(fraud_direct) AS frauds,
    AVG(fraud_direct) AS fraud_rate,
    SUM(CASE WHEN result = 'allow' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS allow_rate,
    SUM(CASE WHEN result = 'review' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS review_rate,
    SUM(CASE WHEN result = 'deny' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS deny_rate
FROM april_base;
""").df()

```

```

overall_fraud_rate = baseline.loc[0, "fraud_rate"]
baseline.to_csv(OUTPUT_DIR / "baseline_april2026.csv", index=False)
print(baseline)

integration_baseline = con.execute("""
SELECT
    integration_point,
    COUNT(*) AS txns,
    SUM(fraud_direct) AS frauds,
    AVG(fraud_direct) AS fraud_rate
FROM april_base
GROUP BY 1
ORDER BY 1;
""").df()
integration_baseline.to_csv(OUTPUT_DIR / "baseline_by_integration_point_april2026.csv", index=False)

# -----
# 5. IPSignal score performance including Score 3
# -----
score_summary = con.execute("""
SELECT
    ipsignal_score,
    CASE
        WHEN ipsignal_score = 5 THEN 'Very High'
        WHEN ipsignal_score = 4 THEN 'High'
        WHEN ipsignal_score = 3 THEN 'Medium-High'
        WHEN ipsignal_score = 2 THEN 'Low'
        WHEN ipsignal_score = 1 THEN 'Very Low'
    END AS risk_level,
    COUNT(*) AS txns,
    SUM(fraud_direct) AS frauds,
    AVG(fraud_direct) AS fraud_rate,
    SUM(CASE WHEN result = 'allow' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS allow_rate,
    SUM(CASE WHEN result = 'review' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS review_rate,
    SUM(CASE WHEN result = 'deny' THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS deny_rate
FROM april_base
GROUP BY 1, 2
ORDER BY ipsignal_score;
""").df()

score_summary["traffic_share"] = score_summary["txns"] / score_summary["txns"].sum()
score_summary["fraud_capture"] = score_summary["frauds"] / score_summary["frauds"].sum()
score_summary["fraud_lift"] = score_summary["fraud_rate"] / overall_fraud_rate
score_summary.to_csv(OUTPUT_DIR / "ipss_distribution_including_score3_april2026.csv", index=False)
print(score_summary)

# Visual 1: separate axes to avoid hiding fraud rate behind traffic share
fig, ax1 = plt.subplots(figsize=(10, 6))
x = np.arange(len(score_summary))
labels = [f"Score {s}" for s in score_summary["ipsignal_score"]]
ax1.bar(x, score_summary["traffic_share"] * 100, width=0.45, label="Traffic Share (%)")
ax1.set_ylabel("Traffic Share (%)")
ax1.set_xticks(x)
ax1.set_xticklabels(labels)
ax2 = ax1.twinx()
ax2.plot(x, score_summary["fraud_rate"] * 100, marker="o", label="Fraud Rate (%)")
ax2.set_ylabel("Fraud Rate (%)")
plt.title("IPSignal Score Distribution and Fraud Rate - April 2026")
fig.legend(loc="upper left", bbox_to_anchor=(0.12, 0.90))
save_plot("01_ipss_distribution_and_fraud_rate_april2026.png")

# Visual 2: Score 3 and Score 4 focus, without Score 5 dominating the chart
score_34 = score_summary[score_summary["ipsignal_score"].isin([3, 4]).copy()]
score_34_combined = pd.DataFrame({
    "ipsignal_score": ["3+4"],
    "risk_level": ["Score 3 + Score 4"],
    "txns": [score_34["txns"].sum()],
    "frauds": [score_34["frauds"].sum()]
})
score_34_combined["fraud_rate"] = score_34_combined["frauds"] / score_34_combined["txns"]
score_34_combined["traffic_share"] = score_34_combined["txns"] / score_summary["txns"].sum()
score_34_combined["fraud_capture"] = score_34_combined["frauds"] / score_summary["frauds"].sum()
score_34_combined["fraud_lift"] = score_34_combined["fraud_rate"] / overall_fraud_rate

score_34_view = pd.concat([
    score_34[["ipsignal_score", "risk_level", "txns", "frauds", "fraud_rate", "traffic_share",
    "fraud_capture", "fraud_lift"]],
    score_34_combined
], ignore_index=True)
score_34_view.to_csv(OUTPUT_DIR / "score_3_4_focused_metrics_april2026.csv", index=False)

metric_plot = score_34_view.copy()
labels = metric_plot["ipsignal_score"].astype(str).map(lambda x: f"Score {x}" if x != "3+4" else "Score 3+4")
x = np.arange(len(metric_plot))
width = 0.22
plt.figure(figsize=(10, 6))
plt.bar(x - width, metric_plot["fraud_rate"] * 100, width, label="Fraud Rate (%)")
plt.bar(x, metric_plot["fraud_lift"], width, label="Fraud Lift (x)")
plt.bar(x + width, metric_plot["fraud_capture"] * 100, width, label="Fraud Capture (%)")
plt.title("Score 3 and Score 4 Focus - April 2026")
plt.xlabel("IPSignal Segment")
plt.ylabel("Metric Value")
plt.xticks(x, labels)

```

```

plt.legend()
save_plot("02_score_3_4_focus_metrics_april2026.png")

# -----
# 6. Reason code taxonomy and reason family performance
# -----
con.execute("""
CREATE OR REPLACE TABLE reason_code_dim AS
SELECT *
FROM (
    VALUES
        ('ANAC', 'Anonymous / privacy'),
        ('ANPR', 'Anonymous / privacy'),
        ('ANOU', 'Anonymous / privacy'),
        ('TR', 'Anonymous / privacy'),

        ('ATK1', 'Attack / bot / spam'),
        ('ATK2', 'Attack / bot / spam'),
        ('BOT_01', 'Attack / bot / spam'),
        ('SPM1', 'Attack / bot / spam'),
        ('SPM2', 'Attack / bot / spam'),

        ('BC', 'Hosting / infrastructure'),
        ('BGP', 'Hosting / infrastructure'),
        ('MGW', 'Hosting / infrastructure'),

        ('SRVR_01', 'Hosting / cloud infrastructure'),
        ('SRVR_02', 'Hosting / cloud infrastructure'),
        ('SRVR_03', 'Hosting / cloud infrastructure'),

        ('CORP_IP', 'Institutional / managed network'),
        ('EDU', 'Institutional / managed network'),
        ('GOVT', 'Institutional / managed network'),
        ('LIB', 'Institutional / managed network'),

        ('CL', 'Home / residential network'),
        ('HF', 'Home / residential network'),
        ('HOM', 'Home / residential network'),

        ('INPR_IP', 'Residential proxy'),
        ('RGPR_IP', 'Residential proxy'),
        ('RP1', 'Residential proxy'),
        ('RP2', 'Residential proxy'),
        ('RP3', 'Residential proxy'),

        ('USGL', 'Geo / location signal'),
        ('USGH', 'Geo / location signal'),

        ('ML1', 'Behavioral intelligence'),
        ('ML2', 'Behavioral intelligence'),

        ('WL_RSK', 'Watchlist / rule list'),
        ('WL_RL', 'Watchlist / rule list')
) AS t(reason_code_norm, reason_family);
""")

con.execute("""
CREATE OR REPLACE TABLE april_reason_long AS
SELECT
    uuid,
    ts,
    integration_point,
    device_id,
    account_code,
    result,
    fraud_direct,
    ipsignal_score,
    reason_code_raw,
    REGEXP_REPLACE(UPPER(TRIM(CAST(reason_code_raw AS VARCHAR))), '[\s\\-]+', '_', 'g') AS
    reason_code_norm
FROM april_base
CROSS JOIN UNNEST([code_1, code_2, code_3, code_4, code_5, code_6]) AS t(reason_code_raw)
WHERE reason_code_raw IS NOT NULL
    AND UPPER(TRIM(CAST(reason_code_raw AS VARCHAR))) NOT IN ('NONE', 'NULL', '', 'NAN');
""")

con.execute("""
CREATE OR REPLACE TABLE april_reason_enriched AS
SELECT
    r.*,
    COALESCE(d.reason_family, 'Unmapped / Needs Review') AS reason_family
FROM april_reason_long r
LEFT JOIN reason_code_dim d
    ON r.reason_code_norm = d.reason_code_norm;
""")

reason_family_mapping = con.execute("""
SELECT * FROM reason_code_dim ORDER BY reason_family, reason_code_norm;
""").df()
reason_family_mapping.to_csv(OUTPUT_DIR / "supplemental_reason_family_groupings.csv", index=False)

reason_family_summary = con.execute("""
WITH family_txn AS (
    SELECT DISTINCT
""")

```

```

        uuid,
        reason_family
    FROM april_reason_enriched
)
SELECT
    f.reason_family,
    COUNT(DISTINCT f.uuid) AS txns,
    SUM(b.fraud_direct) AS frauds,
    AVG(b.fraud_direct) AS fraud_rate,
    COUNT(DISTINCT f.uuid) * 1.0 / (SELECT COUNT(*) FROM april_base) AS traffic_share,
    SUM(b.fraud_direct) * 1.0 / NULLIF((SELECT SUM(fraud_direct) FROM april_base), 0) AS fraud_capture
FROM family_txn f
JOIN april_base b USING (uuid)
GROUP BY 1
HAVING COUNT(DISTINCT f.uuid) >= 30
ORDER BY fraud_rate DESC;
""").df()
reason_family_summary["fraud_lift"] = reason_family_summary["fraud_rate"] / overall_fraud_rate
reason_family_summary.to_csv(OUTPUT_DIR / "reason_family_performance_april2026.csv", index=False)
print(reason_family_summary)

plot_df = reason_family_summary.sort_values("fraud_lift", ascending=True)
plt.figure(figsize=(10, 7))
plt.barh(plot_df["reason_family"], plot_df["fraud_lift"])
plt.axvline(1, linestyle="--")
plt.title("Reason Family Fraud Lift - April 2026")
plt.xlabel("Fraud Lift vs Baseline")
plt.ylabel("Reason Family")
save_plot("03_reason_family_fraud_lift_april2026.png")

# -----
# 7. Reason family flags and RP tiers
# -----
con.execute("""
CREATE OR REPLACE TABLE april_reason_flags AS
SELECT
    uuid,
    MAX(CASE WHEN reason_family = 'Anonymous / privacy' THEN 1 ELSE 0 END) AS has_anonymous_privacy,
    MAX(CASE WHEN reason_family = 'Attack / bot / spam' THEN 1 ELSE 0 END) AS has_attack_bot_spam,
    MAX(CASE WHEN reason_family = 'Residential proxy' THEN 1 ELSE 0 END) AS has_residential_proxy,
    MAX(CASE WHEN reason_family IN ('Hosting / infrastructure', 'Hosting / cloud infrastructure') THEN 1
        ELSE 0 END) AS has_hosting_infra,
    MAX(CASE WHEN reason_family = 'Behavioral intelligence' THEN 1 ELSE 0 END) AS has_behavioral_intel,
    MAX(CASE WHEN reason_family = 'Watchlist / rule list' THEN 1 ELSE 0 END) AS has_watchlist,

    MAX(CASE WHEN reason_code_norm = 'RP1' THEN 1 ELSE 0 END) AS has_rp1,
    MAX(CASE WHEN reason_code_norm = 'RP2' THEN 1 ELSE 0 END) AS has_rp2,
    MAX(CASE WHEN reason_code_norm = 'RP3' THEN 1 ELSE 0 END) AS has_rp3,

    CASE
        WHEN MAX(CASE WHEN reason_code_norm = 'RP3' THEN 1 ELSE 0 END) = 1 THEN 3
        WHEN MAX(CASE WHEN reason_code_norm = 'RP2' THEN 1 ELSE 0 END) = 1 THEN 2
        WHEN MAX(CASE WHEN reason_code_norm = 'RP1' THEN 1 ELSE 0 END) = 1 THEN 1
        ELSE 0
    END AS max_rp_tier,

    COUNT(DISTINCT reason_code_norm) AS number_reason_codes
FROM april_reason_enriched
GROUP BY uuid;
""")

con.execute("""
CREATE OR REPLACE TABLE april_enriched AS
SELECT
    b.*,
    COALESCE(f.has_anonymous_privacy, 0) AS has_anonymous_privacy,
    COALESCE(f.has_attack_bot_spam, 0) AS has_attack_bot_spam,
    COALESCE(f.has_residential_proxy, 0) AS has_residential_proxy,
    COALESCE(f.has_hosting_infra, 0) AS has_hosting_infra,
    COALESCE(f.has_behavioral_intel, 0) AS has_behavioral_intel,
    COALESCE(f.has_watchlist, 0) AS has_watchlist,
    COALESCE(f.has_rp1, 0) AS has_rp1,
    COALESCE(f.has_rp2, 0) AS has_rp2,
    COALESCE(f.has_rp3, 0) AS has_rp3,
    COALESCE(f.max_rp_tier, 0) AS max_rp_tier,
    COALESCE(f.number_reason_codes, 0) AS number_reason_codes
FROM april_base b
LEFT JOIN april_reason_flags f USING (uuid);
""")

# RP summary by integration point. Do not assume RP3 is highest risk.
rp_summary = con.execute("""
WITH universe AS (
    SELECT integration_point, AVG(fraud_direct) AS baseline_rate
    FROM april_enriched
    GROUP BY 1
)
SELECT
    a.integration_point,
    a.max_rp_tier,
    COUNT(*) AS txns,
    SUM(a.fraud_direct) AS frauds,
    AVG(a.fraud_direct) AS fraud_rate,
    COUNT(*) * 1.0 / SUM(COUNT(*)) OVER (PARTITION BY a.integration_point) AS rp_traffic_share,

```

```

SUM(a.fraud_direct) * 1.0 / NULLIF(SUM(SUM(a.fraud_direct)) OVER (PARTITION BY a.integration_point),
0) AS rp_fraud_capture,
AVG(a.ipsignal_score) AS avg_score,
AVG(a.fraud_direct) / NULLIF(MAX(u.baseline_rate), 0) AS fraud_lift_vs_integration_baseline
FROM april_enriched a
JOIN universe u USING (integration_point)
WHERE a.max_rp_tier > 0
GROUP BY 1, 2
ORDER BY 1, 2;
""").df()
rp_summary.to_csv(OUTPUT_DIR / "residential_proxy_tier_performance_april2026.csv", index=False)
print(rp_summary)

```

```

for integration in rp_summary["integration_point"].dropna().unique():
    temp = rp_summary[rp_summary["integration_point"] == integration].copy()
    if temp.empty or temp["frauds"].sum() == 0:
        print(f"No meaningful RP fraud signal for {integration}; skipping plot.")
        continue
    plt.figure(figsize=(8, 5))
    plt.bar(temp["max_rp_tier"].astype(str), temp["fraud_lift_vs_integration_baseline"])
    plt.axhline(1, linestyle="--")
    plt.title(f"Residential Proxy Fraud Lift by RP Tier - {integration}")
    plt.xlabel("Max RP Tier")
    plt.ylabel("Fraud Lift vs Integration Baseline")
    save_plot(f"04_rp_tier_fraud_lift_{integration}.png")

```

```

# -----
# 8. Velocity metrics by integration point
# -----
con.execute("""
CREATE OR REPLACE TABLE april_velocity AS
SELECT
*,
COUNT(*) OVER (
    PARTITION BY integration_point, device_id
    ORDER BY ts
    RANGE BETWEEN INTERVAL 5 MINUTE PRECEDING AND CURRENT ROW
) AS device_txn_5m,

COUNT(*) OVER (
    PARTITION BY integration_point, device_id
    ORDER BY ts
    RANGE BETWEEN INTERVAL 15 MINUTE PRECEDING AND CURRENT ROW
) AS device_txn_15m,

COUNT(*) OVER (
    PARTITION BY integration_point, device_id
    ORDER BY ts
    RANGE BETWEEN INTERVAL 1 HOUR PRECEDING AND CURRENT ROW
) AS device_txn_1h,

COUNT(*) OVER (
    PARTITION BY integration_point, device_id
    ORDER BY ts
    RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW
) AS device_txn_24h,

COUNT(*) OVER (
    PARTITION BY integration_point, account_code
    ORDER BY ts
    RANGE BETWEEN INTERVAL 5 MINUTE PRECEDING AND CURRENT ROW
) AS account_txn_5m,

COUNT(*) OVER (
    PARTITION BY integration_point, real_ipaddress
    ORDER BY ts
    RANGE BETWEEN INTERVAL 5 MINUTE PRECEDING AND CURRENT ROW
) AS ip_txn_5m
FROM april_enriched;
""")

```

```

velocity_features = [
    "device_txn_5m",
    "device_txn_15m",
    "device_txn_1h",
    "device_txn_24h",
    "account_txn_5m",
    "ip_txn_5m",
]

```

```

threshold_queries = []
for feature in velocity_features:
    threshold_queries.append(f"""
SELECT
    integration_point,
    '{feature}' AS velocity_feature,
    QUANTILE_CONT({feature}, 0.90) AS p90_threshold,
    QUANTILE_CONT({feature}, 0.95) AS p95_threshold,
    QUANTILE_CONT({feature}, 0.99) AS p99_threshold
FROM april_velocity
GROUP BY 1
""")

```

```

velocity_thresholds = con.execute(" UNION ALL ".join(threshold_queries)).df()

```

```

velocity_thresholds.to_csv(OUTPUT_DIR / "velocity_thresholds_by_integration_point.csv", index=False)
print(velocity_thresholds)

# -----
# 9. Velocity-only rule backtest with filters
# -----
velocity_rule_results = []
for _, row in velocity_thresholds.iterrows():
    integration = row["integration_point"]
    feature = row["velocity_feature"]
    for threshold_name in ["p90_threshold", "p95_threshold", "p99_threshold"]:
        threshold_value = row[threshold_name]
        if pd.isna(threshold_value):
            continue

        rule_name = f"Velocity | {integration} | {feature} >= {threshold_name}"
        query = f"""
        WITH universe AS (
            SELECT *
            FROM april_velocity
            WHERE integration_point = '{esc_sql(integration)}'
        ),
        triggered AS (
            SELECT *
            FROM universe
            WHERE {feature} >= {threshold_value}
        ),
        u AS (
            SELECT
                COUNT(*) AS total_txns,
                SUM(fraud_direct) AS total_frauds,
                AVG(fraud_direct) AS baseline_rate
            FROM universe
        )
        SELECT
            '{esc_sql(integration)}' AS integration_point,
            '{esc_sql(rule_name)}' AS rule_name,
            'Velocity only' AS rule_type,
            '{feature}' AS velocity_feature,
            '{threshold_name}' AS threshold_type,
            {threshold_value} AS threshold_value,
            COUNT(*) AS triggered_txns,
            COUNT(DISTINCT uuid) AS triggered_unique_txns,
            COUNT(DISTINCT device_id) AS unique_devices,
            COUNT(DISTINCT account_code) AS unique_accounts,
            SUM(fraud_direct) AS triggered_frauds,
            AVG(fraud_direct) AS precision_fraud_rate,
            COUNT(*) * 1.0 / NULLIF(MAX(u.total_txns), 0) AS triggered_txn_share,
            COUNT(*) * 1.0 / NULLIF(MAX(u.total_txns), 0) AS friction_rate,
            SUM(fraud_direct) * 1.0 / NULLIF(MAX(u.total_frauds), 0) AS fraud_capture_rate,
            AVG(fraud_direct) / NULLIF(MAX(u.baseline_rate), 0) AS fraud_lift
        FROM triggered, u;
        """
        velocity_rule_results.append(con.execute(query).df())

velocity_rule_results = pd.concat(velocity_rule_results, ignore_index=True)
velocity_rule_results["legitimate_txns_touched"] = velocity_rule_results["triggered_txns"] -
    velocity_rule_results["triggered_frauds"]
velocity_rule_results.to_csv(OUTPUT_DIR / "velocity_rule_backtest_all.csv", index=False)

# Qualification filters: remove rules with threshold=1, zero fraud, excessive traffic, or weak lift.
qualified_velocity_rules = velocity_rule_results[
    (velocity_rule_results["threshold_value"] > 1) &
    (velocity_rule_results["triggered_txns"] >= MIN_TRIGGERED_TXNS) &
    (velocity_rule_results["triggered_frauds"] >= MIN_TRIGGERED_FRAUDS) &
    (velocity_rule_results["fraud_lift"] >= MIN_FRAUD_LIFT) &
    (velocity_rule_results["friction_rate"] <= MAX_FRICTION_RATE) &
    (velocity_rule_results["triggered_txn_share"] <= MAX_TRIGGERED_SHARE)
].copy()

qualified_velocity_rules = qualified_velocity_rules.sort_values(
    ["integration_point", "fraud_lift", "fraud_capture_rate"],
    ascending=[True, False, False]
)
qualified_velocity_rules.to_csv(OUTPUT_DIR / "qualified_velocity_rules_by_integration_point.csv",
    index=False)

# Top velocity rules by integration point
top_velocity_rules = (
    qualified_velocity_rules
    .groupby("integration_point", group_keys=False)
    .head(MAX_RULES_PER_INTEGRATION)
    .reset_index(drop=True)
)
top_velocity_rules.to_csv(OUTPUT_DIR / "recommended_velocity_rules_by_integration_point.csv", index=False)
print(top_velocity_rules)

for integration in top_velocity_rules["integration_point"].dropna().unique():
    temp = top_velocity_rules[top_velocity_rules["integration_point"] ==
        integration].sort_values("fraud_lift", ascending=True)
    if temp.empty:
        print(f"No qualified velocity rules for {integration}")
        continue
    plt.figure(figsize=(10, 6))

```

```

plt.barh(temp["rule_name"], temp["fraud_lift"])
plt.axvline(1, linestyle="--")
plt.title(f'Recommended Velocity Rules - {integration}')
plt.xlabel("Fraud Lift")
plt.ylabel("Rule Name")
save_plot(f"05_recommended_velocity_rules_{integration}.png")

# -----
# 10. Layered candidate rules: score + reason + selected velocity
# -----
# Note: RP rules now evaluate RP1/RP2/RP3 separately.
# Note: application and receiver rules with zero fraud will be filtered out.

# Pull useful thresholds for dynamic layered velocity conditions.
thr = velocity_thresholds.copy()
def get_threshold(integration, feature, threshold_col, default_value):
    m = thr[(thr["integration_point"] == integration) & (thr["velocity_feature"] == feature)]
    if m.empty or pd.isna(m.iloc[0][threshold_col]):
        return default_value
    return m.iloc[0][threshold_col]

integrations = con.execute("SELECT DISTINCT integration_point FROM april_velocity ORDER BY
1").df()["integration_point"].tolist()

rule_specs = []

# Static layered rules applied to all integration points.
static_rules = [
    ("Score 3 + Anonymous Privacy", "Layered", 3, "ipsignal_score = 3 AND has_anonymous_privacy = 1",
     "Review / step-up candidate"),
    ("Score 4 + Anonymous Privacy", "Layered", 3, "ipsignal_score = 4 AND has_anonymous_privacy = 1",
     "Review / step-up candidate"),
    ("Score 3/4 + Anonymous Privacy", "Layered", 4, "ipsignal_score IN (3, 4) AND has_anonymous_privacy =
     1", "Review / step-up candidate"),
    ("Score 3/4 + Attack Bot Spam", "Layered", 4, "ipsignal_score IN (3, 4) AND has_attack_bot_spam = 1",
     "Review / step-up candidate"),
    ("Score 3/4 + Residential Proxy", "Layered", 5, "ipsignal_score IN (3, 4) AND has_residential_proxy =
     1", "Review candidate"),
    ("Score 3/4 + Hosting Infra", "Layered", 6, "ipsignal_score IN (3, 4) AND has_hosting_infra = 1",
     "Monitor / secondary signal"),
    ("Score 5 + Anonymous Privacy", "Layered", 4, "ipsignal_score = 5 AND has_anonymous_privacy = 1",
     "Review / step-up candidate"),
    ("Score 5 + Attack Bot Spam", "Layered", 5, "ipsignal_score = 5 AND has_attack_bot_spam = 1", "Review
     candidate"),
    ("Score 4 + RP1", "Layered", 5, "ipsignal_score = 4 AND has_rp1 = 1", "Deny candidate if friction
     tolerance allows"),
    ("Score 4 + RP2", "Layered", 5, "ipsignal_score = 4 AND has_rp2 = 1", "Deny candidate if friction
     tolerance allows"),
    ("Score 4 + RP3", "Layered", 5, "ipsignal_score = 4 AND has_rp3 = 1", "Deny candidate if friction
     tolerance allows"),
]

for integration in integrations:
    for rule_name, rule_type, priority, condition, action in static_rules:
        rule_specs.append({
            "integration_point": integration,
            "rule_name": rule_name,
            "rule_type": rule_type,
            "rule_priority": priority,
            "recommended_action": action,
            "condition": f"integration_point = '{esc_sql(integration)}' AND {condition}",
        })

# Dynamic layered velocity rules by integration point.
t_1h = get_threshold(integration, "device_txn_1h", "p99_threshold", 6)
t_24h = get_threshold(integration, "device_txn_24h", "p99_threshold", 21)
t_15m = get_threshold(integration, "device_txn_15m", "p99_threshold", 5)

# Only add meaningful thresholds > 1.
if t_1h > 1:
    rule_specs.append({
        "integration_point": integration,
        "rule_name": f"Score 3/4 + High Device Velocity 1h",
        "rule_type": "Layered + velocity",
        "rule_priority": 7,
        "recommended_action": "Review / step-up candidate",
        "condition": f"integration_point = '{esc_sql(integration)}' AND ipsignal_score IN (3, 4) AND
         device_txn_1h >= {t_1h}",
    })
if t_24h > 1:
    rule_specs.append({
        "integration_point": integration,
        "rule_name": f"Score 3/4 + High Device Velocity 24h",
        "rule_type": "Layered + velocity",
        "rule_priority": 7,
        "recommended_action": "Review / step-up candidate",
        "condition": f"integration_point = '{esc_sql(integration)}' AND ipsignal_score IN (3, 4) AND
         device_txn_24h >= {t_24h}",
    })
if t_15m > 1:
    rule_specs.append({
        "integration_point": integration,
        "rule_name": f"Score 3/4 + High Device Velocity 15m",
        "rule_type": "Layered + velocity",
    })

```

```

        "rule_priority": 8,
        "recommended_action": "Review candidate",
        "condition": f'integration_point = '{esc_sql(integration)}' AND ipsignal_score IN (3, 4) AND
            device_txn_15m >= {t_15m}',
    })

# Build rule_hits table.
selects = []
for spec in rule_specs:
    selects.append(f"""
    SELECT DISTINCT
        uuid,
        integration_point,
        '{esc_sql(spec['rule_name'])}' AS rule_name,
        '{esc_sql(spec['rule_type'])}' AS rule_type,
        {int(spec['rule_priority'])} AS rule_priority,
        '{esc_sql(spec['recommended_action'])}' AS recommended_action
    FROM april_velocity
    WHERE {spec['condition']}
    """)

con.execute("CREATE OR REPLACE TABLE rule_hits AS " + " UNION ALL ".join(selects))

# Backtest layered and layered+velocity rules.
rule_backtest = con.execute("""
WITH universe AS (
    SELECT
        integration_point,
        COUNT(*) AS total_txns,
        SUM(fraud_direct) AS total_frauds,
        AVG(fraud_direct) AS baseline_rate
    FROM april_velocity
    GROUP BY 1
),
hit_unique AS (
    SELECT DISTINCT
        uuid,
        integration_point,
        rule_name,
        rule_type,
        rule_priority,
        recommended_action
    FROM rule_hits
),
joined AS (
    SELECT
        h.integration_point,
        h.rule_name,
        h.rule_type,
        h.rule_priority,
        h.recommended_action,
        v.uuid,
        v.fraud_direct,
        v.device_id,
        v.account_code
    FROM hit_unique h
    JOIN april_velocity v USING (uuid, integration_point)
)
SELECT
    j.integration_point,
    j.rule_name,
    j.rule_type,
    MAX(j.rule_priority) AS rule_priority,
    MAX(j.recommended_action) AS recommended_action,
    COUNT(DISTINCT j.uuid) AS triggered_txns,
    SUM(j.fraud_direct) AS triggered_frauds,
    AVG(j.fraud_direct) AS precision_fraud_rate,
    COUNT(DISTINCT j.uuid) * 1.0 / NULLIF(MAX(u.total_txns), 0) AS triggered_txn_share,
    SUM(j.fraud_direct) * 1.0 / NULLIF(MAX(u.total_frauds), 0) AS fraud_capture_rate,
    AVG(j.fraud_direct) / NULLIF(MAX(u.baseline_rate), 0) AS fraud_lift,
    COUNT(DISTINCT j.uuid) * 1.0 / NULLIF(MAX(u.total_txns), 0) AS friction_rate,
    COUNT(DISTINCT j.device_id) AS unique_devices,
    COUNT(DISTINCT j.account_code) AS unique_accounts,
    COUNT(DISTINCT j.uuid) - SUM(j.fraud_direct) AS legitimate_txns_touched
FROM joined j
JOIN universe u USING (integration_point)
GROUP BY 1, 2, 3
ORDER BY 1, fraud_lift DESC, fraud_capture_rate DESC;
""").df()

rule_backtest.to_csv(OUTPUT_DIR / "layered_rule_backtest_all.csv", index=False)
print(rule_backtest.head(50))

# Qualification filters for layered rules.
qualified_layered_rules = rule_backtest[
    (rule_backtest["triggered_txns"] >= MIN_TRIGGERED_TXNS) &
    (rule_backtest["triggered_frauds"] >= MIN_TRIGGERED_FRAUDS) &
    (rule_backtest["fraud_lift"] >= MIN_FRAUD_LIFT) &
    (rule_backtest["friction_rate"] <= MAX_FRICTION_RATE) &
    (rule_backtest["triggered_txn_share"] <= MAX_TRIGGERED_SHARE)
].copy()
qualified_layered_rules.to_csv(OUTPUT_DIR / "qualified_layered_rules_by_integration_point.csv",
    index=False)

# Combine qualified layered and velocity-only rules.

```

```

# Align columns first.
velocity_for_reco = top_velocity_rules.rename(columns={
    "threshold_type": "rule_threshold_type",
    "threshold_value": "rule_threshold_value",
}).copy()
velocity_for_reco["rule_priority"] = 9
velocity_for_reco["recommended_action"] = "Review / step-up candidate"

needed_cols = [
    "integration_point", "rule_name", "rule_type", "rule_priority", "recommended_action",
    "triggered_txns", "triggered_frauds", "precision_fraud_rate", "triggered_txn_share",
    "fraud_capture_rate", "fraud_lift", "friction_rate", "unique_devices", "unique_accounts",
    "legitimate_txns_touched"
]

combined_recommendations = pd.concat([
    qualified_layered_rules[needed_cols],
    velocity_for_reco[needed_cols]
], ignore_index=True)

combined_recommendations = combined_recommendations.sort_values(
    ["integration_point", "fraud_lift", "fraud_capture_rate"],
    ascending=[True, False, False]
)
combined_recommendations.to_csv(OUTPUT_DIR / "combined_qualified_rule_recommendations.csv", index=False)

# Top recommendations by integration point.
top_recommendations = (
    combined_recommendations
    .groupby("integration_point", group_keys=False)
    .head(MAX_RULES_PER_INTEGRATION)
    .reset_index(drop=True)
)
top_recommendations.to_csv(OUTPUT_DIR / "top_recommended_rules_by_integration_point.csv", index=False)
print(top_recommendations)

for integration in top_recommendations["integration_point"].dropna().unique():
    temp = top_recommendations[top_recommendations["integration_point"] ==
        integration].sort_values("fraud_lift", ascending=True)
    if temp.empty:
        print(f"No recommended rules met criteria for {integration}")
        continue
    plt.figure(figsize=(10, 6))
    plt.barh(temp["rule_name"], temp["fraud_lift"])
    plt.axvline(1, linestyle="-.")
    plt.title(f"Top Recommended Rules by Fraud Lift - {integration}")
    plt.xlabel("Fraud Lift")
    plt.ylabel("Rule Name")
    save_plot(f"06_top_recommended_rules_{integration}.png")

# -----
# 11. Deduplicated cumulative and incremental rule impact
# -----
# Only use qualified recommendations; do not calculate cumulative impact for rules with zero value.

# Create a combined hit table for all qualified recommendations.
qualified_names = top_recommendations[["integration_point", "rule_name"]].drop_duplicates()
qualified_names.to_csv(OUTPUT_DIR / "rules_used_for_cumulative_impact.csv", index=False)

# Generate rule hit tables for velocity-only recommendations also.
velocity_hit_selects = []
for _, row in top_velocity_rules.iterrows():
    integration = row["integration_point"]
    rule_name = row["rule_name"]
    feature = row["velocity_feature"]
    threshold = row["threshold_value"]
    velocity_hit_selects.append(f"""
SELECT DISTINCT
    uuid,
    integration_point,
    '{esc_sql(rule_name)}' AS rule_name
FROM april_velocity
WHERE integration_point = '{esc_sql(integration)}'
AND {feature} >= {threshold}
""")

# Layered rule_hits already exist. Build a unified hit table.
base_hit_select = "SELECT DISTINCT uuid, integration_point, rule_name FROM rule_hits"
all_hit_sql = base_hit_select
if velocity_hit_selects:
    all_hit_sql += " UNION ALL " + " UNION ALL ".join(velocity_hit_selects)

con.execute("CREATE OR REPLACE TABLE all_recommended_rule_hits AS " + all_hit_sql)

# Keep only top recommended rules.
con.register("top_recommendations_df", top_recommendations[["integration_point", "rule_name",
    "fraud_lift", "fraud_capture_rate", "friction_rate"]])
con.execute("""
CREATE OR REPLACE TABLE selected_rule_hits AS
SELECT DISTINCT h.uuid, h.integration_point, h.rule_name
FROM all_recommended_rule_hits h
JOIN top_recommendations_df r
    ON h.integration_point = r.integration_point
    AND h.rule_name = r.rule_name;

```

```

"""
cumulative_rows = []
for integration in top_recommendations["integration_point"].dropna().unique():
    temp_rules = top_recommendations[top_recommendations["integration_point"] == integration].copy()
    if temp_rules.empty:
        continue

    # Order by lift, then fraud capture.
    temp_rules = temp_rules.sort_values(["fraud_lift", "fraud_capture_rate"], ascending=[False,
False]).reset_index(drop=True)
    previous_uuids = set()

    for i in range(1, len(temp_rules) + 1):
        selected = temp_rules.iloc[:i]
        selected_rule_names = selected["rule_name"].tolist()
        selected_rule_sql = ",".join([f'"{esc_sql(x)}"' for x in selected_rule_names])

        current_df = con.execute(f"""
SELECT DISTINCT h.uuid
FROM selected_rule_hits h
WHERE h.integration_point = '{esc_sql(integration)}'
AND h.rule_name IN ({selected_rule_sql});
""").df()
        current_uuids = set(current_df["uuid"].tolist())
        incremental_uuids = current_uuids - previous_uuids

        if not current_uuids:
            continue

        current_uuid_sql = ",".join([f'"{esc_sql(str(x))}"' for x in current_uuids])
        incremental_uuid_sql = ",".join([f'"{esc_sql(str(x))}"' for x in incremental_uuids]) if
incremental_uuids else "NULL"

        metrics = con.execute(f"""
WITH universe AS (
    SELECT *
    FROM april_velocity
    WHERE integration_point = '{esc_sql(integration)}'
),
triggered AS (
    SELECT *
    FROM universe
    WHERE CAST(uuid AS VARCHAR) IN ({current_uuid_sql})
),
incr AS (
    SELECT *
    FROM universe
    WHERE CAST(uuid AS VARCHAR) IN ({incremental_uuid_sql})
),
u AS (
    SELECT COUNT(*) AS total_txns, SUM(fraud_direct) AS total_frauds, AVG(fraud_direct) AS
baseline_rate
    FROM universe
)
SELECT
COUNT(*) AS dedup_triggered_txns,
SUM(fraud_direct) AS dedup_triggered_frauds,
AVG(fraud_direct) AS precision_fraud_rate,
COUNT(*) * 1.0 / NULLIF(MAX(u.total_txns), 0) AS triggered_txn_share,
SUM(fraud_direct) * 1.0 / NULLIF(MAX(u.total_frauds), 0) AS fraud_capture_rate,
COUNT(*) * 1.0 / NULLIF(MAX(u.total_txns), 0) AS friction_rate,
COUNT(DISTINCT device_id) AS unique_devices,
COUNT(DISTINCT account_code) AS unique_accounts,
COUNT(*) - SUM(fraud_direct) AS legitimate_txns_touched,
(SELECT COUNT(*) FROM incr) AS incremental_triggered_txns,
(SELECT SUM(fraud_direct) FROM incr) AS incremental_triggered_frauds
FROM triggered, u;
""").df().iloc[0].to_dict()

        cumulative_rows.append({
            "integration_point": integration,
            "scenario_number": i,
            "rules_included": " + ".join(selected_rule_names),
            **metrics,
        })
        previous_uuids = current_uuids

cumulative_rule_impact = pd.DataFrame(cumulative_rows)
if not cumulative_rule_impact.empty:
    cumulative_rule_impact.to_csv(OUTPUT_DIR / "deduplicated_cumulative_rule_impact.csv", index=False)
    print(cumulative_rule_impact.head(20))

    for integration in cumulative_rule_impact["integration_point"].dropna().unique():
        temp = cumulative_rule_impact[cumulative_rule_impact["integration_point"] == integration]
        if temp.empty or temp["dedup_triggered_frauds"].sum() == 0:
            print(f"Skipping cumulative plot for {integration}; no captured fraud.")
            continue
        plt.figure(figsize=(9, 5))
        plt.plot(temp["scenario_number"], temp["fraud_capture_rate"] * 100, marker="o", label="Fraud
Capture %")
        plt.plot(temp["scenario_number"], temp["friction_rate"] * 100, marker="o", label="Friction %")
        plt.title(f"Deduplicated Cumulative Rule Impact - {integration}")
        plt.xlabel("Number of Rules Included")
        plt.ylabel("Percentage")

```

```

plt.legend()
save_plot(f"07_cumulative_rule_impact_{integration}.png")
else:
    print("No qualified rules available for cumulative impact.")

# -----
# 12. Final recommendation summary
# -----
# Create a client-facing rule recommendation table.
final_summary = top_recommendations.copy()
final_summary["recommendation_note"] = np.select(
    [
        final_summary["rule_type"].eq("Velocity only"),
        final_summary["rule_name"].str.contains("Anonymous Privacy", case=False, na=False),
        final_summary["rule_name"].str.contains("RP", case=False, na=False),
    ],
    [
        "Velocity-only rule; useful where behavior is stronger than score/reason combinations.",
        "High precision reason-family signal; prioritize for review or step-up.",
        "RP tier rule; validate by integration point because RP risk is not uniform.",
    ],
    default="Layered rule candidate; validate operational friction before deployment."
)
final_summary.to_csv(OUTPUT_DIR / "final_client_facing_rule_recommendations.csv", index=False)

# A compact table for deck use.
deck_columns = [
    "integration_point", "rule_name", "rule_type", "recommended_action", "triggered_txns",
    "triggered_txn_share", "triggered_frauds", "precision_fraud_rate", "fraud_lift",
    "fraud_capture_rate", "friction_rate", "legitimate_txns_touched", "recommendation_note"
]
final_summary[deck_columns].to_csv(OUTPUT_DIR / "deck_rule_recommendation_table.csv", index=False)
print(final_summary[deck_columns])

print("Completed. Outputs saved to:", OUTPUT_DIR)

```

Expected Outputs

Output	Purpose
baseline_april2026.csv	Overall April scored-universe baseline
baseline_by_integration_point_april2026.csv	Baseline by integration point
ipss_distribution_including_score3_april2026.csv	Score 1-5 metrics including Score 3
score_3_4_focused_metrics_april2026.csv	Score 3, Score 4, and combined Score 3+4 view
supplemental_reason_family_groupings.csv	Reason code to reason family mapping
reason_family_performance_april2026.csv	Reason family performance
residential_proxy_tier_performance_april2026.csv	RP1/RP2/RP3 performance by integration point
velocity_thresholds_by_integration_point.csv	P90/P95/P99 thresholds by integration point
velocity_rule_backtest_all.csv	All velocity rules before filters
qualified_velocity_rules_by_integration_point.csv	Velocity rules that meet criteria
layered_rule_backtest_all.csv	All layered rules before filters
qualified_layered_rules_by_integration_point.csv	Layered rules that meet criteria
combined_qualified_rule_recommendations.csv	Combined velocity and layered recommendations
top_recommended_rules_by_integration_point.csv	Top recommendations by integration point
deduplicated_cumulative_rule_impact.csv	Deduplicated cumulative and incremental rule impact
final_client_facing_rule_recommendations.csv	Final recommendations with interpretation notes
deck_rule_recommendation_table.csv	Compact table for presentation use

Visualization Outputs

- 01_ipss_distribution_and_fraud_rate_april2026.png
- 02_score_3_4_focus_metrics_april2026.png
- 03_reason_family_fraud_lift_april2026.png
- 04_rp_tier_fraud_lift.png
- 05_recommended_velocity_rules.png
- 06_top_recommended_rules.png
- 07_cumulative_rule_impact.png

Final Note

If an integration point has no rules that meet the minimum criteria, the code will not produce an empty chart. Instead, it prints a message and leaves that integration point out of the recommendation chart. This is intentional and avoids presenting weak or misleading recommendations.